

Java Backend Architecture

Interview Series

Chapter 7.7

Gradle Multi-Module & Build Cache

50+ Interview Q&A

Code Examples

Architecture Diagrams

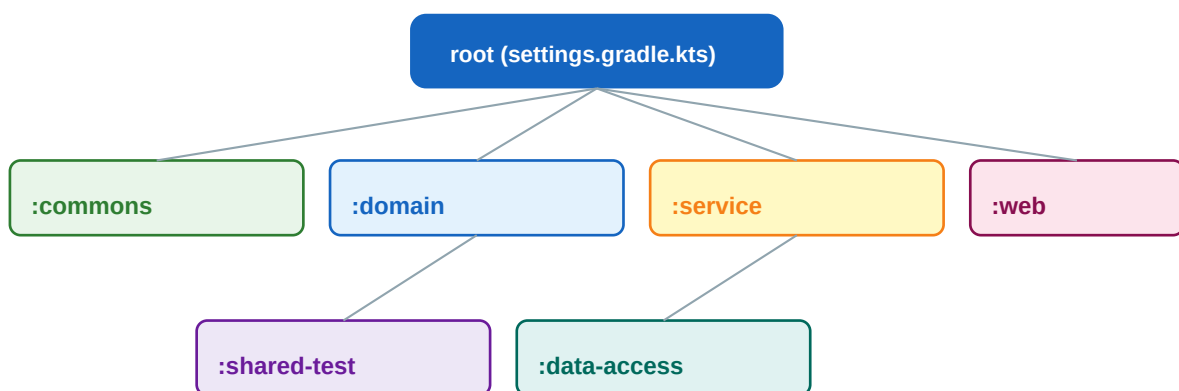
Advanced Topics

Chapter 7.7 — Gradle Multi-Module & Build Cache

Introduction

Gradle's multi-project build support and build caching infrastructure provide dramatic performance improvements for large codebases. The build cache stores task outputs keyed by input hash — allowing identical tasks to be skipped even across different machines or CI runs. The configuration cache avoids re-evaluating build scripts on subsequent runs. Together, these features can reduce build times by 80-90% in large enterprise projects.

Multi-Project Build Structure



Reference subprojects as: `implementation(project(":domain"))`

```
// settings.gradle.kts rootProject.name = "enterprise-app" include( ":commons",
":shared-test", ":domain", ":data-access", ":service", ":web" ) // Optional: custom project
directories project(":web").projectDir = file("modules/web-api") // Plugin management (applied
before build scripts) pluginManagement { repositories { gradlePluginPortal() mavenCentral() } }
// Central dependency resolution management dependencyResolutionManagement {
repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS) repositories { mavenCentral() }
versionCatalogs { create("libs") { from(files("gradle/libs.versions.toml")) } } }
```

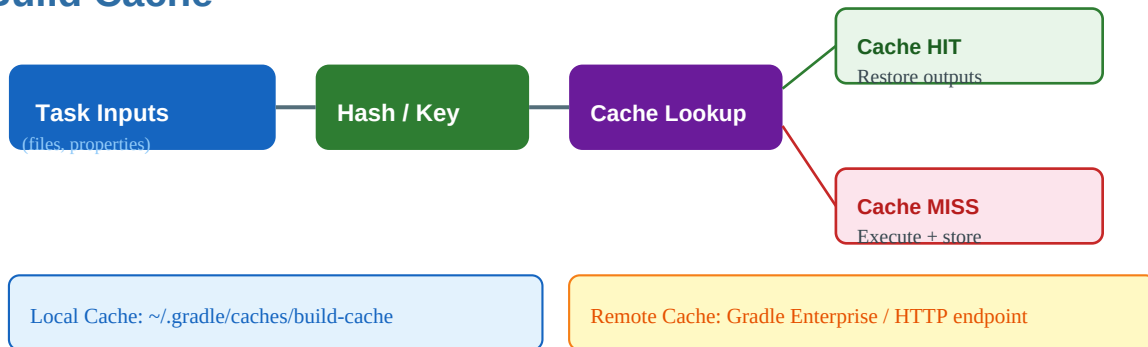
Convention Plugins with buildSrc

```
// buildSrc/src/main/kotlin/java-conventions.gradle.kts // This replaces repetitive
allprojects/subprojects blocks plugins { `java-library` `jacoco` } java { toolchain {
languageVersion = JavaLanguageVersion.of(17) } } tasks.withType<Test> { useJUnitPlatform()
maxParallelForks = Runtime.getRuntime().availableProcessors() / 2 testLogging {
events("passed", "skipped", "failed") } } tasks.jacocoTestReport { dependsOn(tasks.test)
reports { xml.required = true html.required = true } } // Apply this convention plugin in each
subproject: // plugins { id("java-conventions") }
```

Inter-Project Dependencies

```
// service/build.gradle.kts plugins { id("java-conventions") // convention plugin from buildSrc
} dependencies { // Project dependency – Gradle builds :domain first
implementation(project(":domain")) implementation(project(":data-access")) // Test fixtures
dependency testImplementation(testFixtures(project(":domain"))) // External dependencies from
version catalog implementation(libs.spring.boot.starter)
testImplementation(libs.bundles.testing) }
```

Build Cache



The build cache stores task outputs keyed by a hash of all task inputs. If the same task runs with identical inputs (even on a different machine), outputs are restored from cache instead of re-executing. The **local cache** (default) resides in `~/.gradle/caches/build-cache`. The **remote cache** is shared across team members and CI runners.

```
// gradle.properties – enable build cache org.gradle.caching=true // settings.gradle.kts –
configure remote build cache (Gradle Enterprise) buildCache { local { isEnabled = true
removeUnusedEntriesAfterDays = 7 } remote<HttpBuildCache> { url =
uri("https://cache.example.com/cache/") isPush = System.getenv("CI") != null // only CI pushes
to remote credentials { username = System.getenv("CACHE_USER") password =
System.getenv("CACHE_PASSWORD") } } } # Command line: ./gradlew build --build-cache ./gradlew
build --no-build-cache # disable for this run
```

Making Custom Tasks Cacheable

```
@CacheableTask // tells Gradle this task supports build cache abstract class GenerateApiDocs :
DefaultTask() { @get:InputFiles @get:PathSensitive(PathSensitivity.RELATIVE) // REQUIRED for
caching abstract val sourceFiles: ConfigurableFileCollection @get:Input abstract val apiTitle:
Property<String> @get:OutputDirectory abstract val outputDir: DirectoryProperty @TaskAction
fun generate() { // Task implementation outputDir.get().asFile.mkdirs() } } // @PathSensitive
is required – tells Gradle whether file paths matter // RELATIVE: relative path matters (most
common) // NONE: only content matters (path changes dont invalidate cache) // NAME_ONLY: only
filename matters // ABSOLUTE: full path matters (breaks relocatability – avoid!)
```

Configuration Cache

The configuration cache serialises the result of the Configuration phase to disk. On subsequent builds with unchanged build scripts, Gradle skips Configuration entirely and goes straight to Execution. This can save 2-10+ seconds per build in large projects.

```
# Enable configuration cache # gradle.properties org.gradle.configuration-cache=true
org.gradle.configuration-cache.problems=warn # or "fail" # Command line: ./gradlew build
--configuration-cache # First run: "Calculating task graph as no cached configuration is
available" # Subsequent run: "Reusing configuration cache" # Common configuration cache
incompatibilities: # - Reading project.buildDir in doLast {} (use layout.buildDirectory) # -
Referencing non-serializable objects in task configuration # - Using File-returning methods
instead of Provider<File> # - Capturing project reference in a task action
```

Parallel Execution

```
# gradle.properties org.gradle.parallel=true org.gradle.workers.max=8 # or leave default
(number of CPU cores) # Tasks in independent subprojects run concurrently # Worker API for
```

```
parallel task internals: abstract class ProcessFilesTask @Inject constructor( private val
workerExecutor: WorkerExecutor ) : DefaultTask() { @get:InputFiles abstract val files:
ConfigurableFileCollection @TaskAction fun process() { val queue =
workerExecutor.noIsolation() files.forEach { file -> queue.submit(ProcessFile::class) {
inputFile.set(file) } } } }
```

Performance Tuning gradle.properties

```
# gradle.properties — key performance settings # Gradle daemon (default: true since Gradle 3.x)
org.gradle.daemon=true org.gradle.daemon.idletimeout=10800000 # 3 hours # Parallel execution
org.gradle.parallel=true # Build cache org.gradle.caching=true # Configuration cache
org.gradle.configuration-cache=true # JVM memory for the Gradle daemon
org.gradle.jvmargs=-Xmx4g -XX:+HeapDumpOnOutOfMemoryError -Dfile.encoding=UTF-8 # Watch file
system for changes (avoids rescanning on each build) org.gradle.vfs.watch=true # Show test
results in console org.gradle.console=verbose
```

50+ Interview Questions & Answers

Q1. How do you include a subproject in a Gradle multi-project build?

In settings.gradle.kts: include(':module-name'). Gradle looks for the build file in the directory matching the module name. Use project(':module').projectDir to customize the directory.

Q2. How do you declare a project dependency in Gradle?

implementation(project(':domain')). Gradle ensures :domain is built before the current project. The dependency resolves to the :domain project's compile output.

Q3. What is a convention plugin?

A plugin in buildSrc (or an included build) that encapsulates common build configuration. Applied in subprojects via plugins { id('java-conventions') }. Replaces repetitive allprojects/subprojects blocks.

Q4. What is buildSrc?

A special Gradle directory at the root. Gradle compiles and tests it first, making its output available to all build scripts as a plugin. Used for convention plugins, custom tasks, and shared build logic.

Q5. What is the difference between buildSrc and an included build?

buildSrc is private to the current root project. Included builds (composite builds) can be reused across multiple root projects and produce cacheable outputs. Prefer included builds for widely-shared logic.

Q6. What is a composite build?

A build that includes other builds as participants using includeBuild() in settings.gradle. The included build's artifacts can substitute for external dependencies — useful for local development of multi-repo projects.

Q7. What does the Gradle build cache store?

Task outputs (output files and directories) keyed by a hash of all task inputs (input files, input properties, task class implementation hash). Allows skipping tasks when inputs haven't changed.

Q8. What is the difference between up-to-date checks and the build cache?

UP-TO-DATE: task is skipped because nothing changed since the LAST RUN on THIS machine. Build cache: task outputs are restored from cache — works across machines and CI runners, and after clean builds.

Q9. What is the local build cache?

Stored in ~/.gradle/caches/build-cache. Enabled by default when org.gradle.caching=true. Persists across builds on the same machine, including after ./gradlew clean.

Q10. What is the remote build cache?

A shared HTTP cache server (Gradle Enterprise, Develocity, or custom HTTP server) that stores and serves cached outputs to all developers and CI runners, enabling cache hits after another developer or CI built the same code.

Q11. How do you make a custom task cacheable?

Annotate with `@CacheableTask`. Declare all inputs with `@InputFile/@InputFiles/@Input` and outputs with `@OutputFile/@OutputDirectory`. Use `@PathSensitive` on file inputs.

Q12. What is `@PathSensitive`?

Required on `@InputFile/@InputFiles` annotations for cacheable tasks. Tells Gradle how much of the path to include in the cache key: `RELATIVE` (most common), `NONE` (only content), `NAME_ONLY`, `ABSOLUTE` (breaks cache portability).

Q13. Why does `ABSOLUTE` path sensitivity break cache portability?

Different machines have different absolute paths. If the path is part of the cache key, the same code checked out to `/home/user1/project` and `/home/user2/project` would never hit each other's cache entries.

Q14. What is the configuration cache?

Saves the result of the Configuration phase to disk. On subsequent builds with unchanged build scripts, Gradle skips Configuration entirely and uses the cached task graph. Enabled with `org.gradle.configuration-cache=true`.

Q15. What are common configuration cache incompatibilities?

Accessing `project.buildDir` in task actions (use `layout.buildDirectory` provider instead), capturing non-serializable objects in task closures, using `FileCollection` in configuration actions, and accessing `build.gradle` at configuration time.

Q16. What is the Gradle file system watching?

`org.gradle.vfs.watch=true` — Gradle continuously monitors the file system for changes between builds. Avoids full file system scan at the start of each build. Reduces configuration time significantly.

Q17. How does parallel execution work in Gradle multi-project builds?

With `org.gradle.parallel=true`, independent subprojects (no project dependencies between them) build concurrently using a thread pool. Dependent projects still build in order.

Q18. What is the Worker API?

Allows a single task to submit work items that execute in parallel using Gradle's managed thread pool. Work items can run with `nolsolation()`, `classLoaderIsolation()`, or `processIsolation()`. Enables fine-grained parallelism within a task.

Q19. What is test fixtures in Gradle?

A built-in feature (`java-test-fixtures` plugin) that creates a `testFixtures` source set. Other projects can declare: `testImplementation(testFixtures(project(':domain')))`. Avoids test-jar complications from Maven.

Q20. How do you run tasks in a specific subproject?

`./gradlew :service:test` — runs the test task in the `:service` subproject only. `./gradlew :service:test :domain:test` — runs both. The colon prefix identifies the subproject path.

Q21. What is the difference between `./gradlew test` and `./gradlew check`?

`test` runs only the Test tasks. `check` runs all verification tasks: test, plus code quality checks (Checkstyle, PMD, SpotBugs, JaCoCo threshold checks). `check` aggregates all quality gates.

Q22. What is the Gradle scan and how is it used?

`./gradlew build --scan` generates a detailed build scan uploaded to `scans.gradle.com`. Shows timeline, task performance, test results, dependency resolution, and cache performance. Essential for diagnosing slow builds.

Q23. What does `--rerun-tasks` flag do?

Forces all tasks to re-execute even if they are UP-TO-DATE or would hit the build cache. Used to force a full clean rebuild without deleting outputs.

Q24. How do you aggregate test results from all subprojects?

Configure a root-level Test task that depends on all subproject test tasks and aggregates the XML results, or use the `test-report-aggregation` plugin introduced in Gradle 7.4.

Q25. What is the `test-report-aggregation` plugin?

Aggregates JUnit XML test results from all subprojects into a single HTML report. Apply to the root project: `apply plugin: 'test-report-aggregation'`. Run: `./gradlew testAggregateTestReport`.

Q26. What is Gradle's outgoing variants?

Each project exposes outgoing variants (`apiElements`, `runtimeElements`, etc.) that consumers select via attribute matching. Enables a single project to provide different artifacts for different use cases.

Q27. What is the `includeBuild()` function?

In `settings.gradle`: `includeBuild('../other-project')` adds another Gradle build as a composite participant. Gradle substitutes dependencies on that project with its local build output — enables local multi-repo development.

Q28. What is the Gradle Toolchain?

Specifies which JDK version to use for compilation and testing: `java { toolchain { languageVersion = JavaLanguageVersion.of(17) } }`. Gradle auto-provisions the JDK via Foojay Toolchain Convention if not found.

Q29. How do you cache remote build cache entries only from CI?

Set `isPush = System.getenv('CI') != null` in the remote cache config. CI runners write to the cache; developers only read from it. Prevents developer-specific output from polluting the shared cache.

Q30. What is Gradle Enterprise / Develocity?

Gradle's commercial platform providing: remote build cache server, build scan storage and analytics, predictive test selection (runs only tests likely to fail), and acceleration for Gradle and Maven builds.

Q31. What is predictive test selection?

A Gradle Enterprise feature that analyses test history and code changes to identify which tests are likely to fail, running only those tests to reduce test execution time in CI.